Embedded Spatial Measurement System – Technical Reference Manual

Sam Wang | wangs562 | 400504643

COMP ENG 2DX3

Dr. Yaser Haddara, Dr. Mohamed Elamien, Dr. Shahrukh Athar

2026 April 7

As a future member of the engineering profession, the student is responsible for performing the required work in an honest manner, without plagiarism and cheating. Submitting this work with my name and student number is a statement and understanding that this work is our own and adheres to the Academic Integrity Policy of McMaster University and the Code of Conduct of the Professional Engineers of Ontario. **[Sam Wang, wangs562, 400504643]**

Table of Contents:

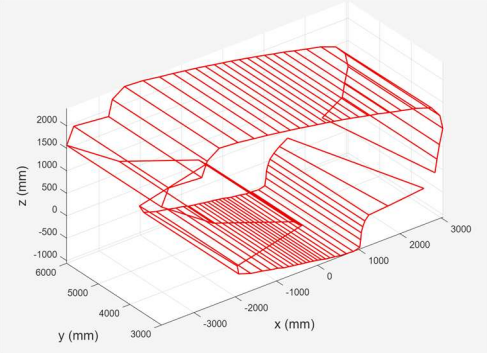
Table of Contents: 1
1 – Device Overview: 3
1.1 – General Description: 3
1.2 – Purposes: 3
1.3 – Terminology: 3
2 – Product Overview 4
2.1 – Technical Specifications 4
2.2 – System Operating Modes 4
2.3 – System Settings and Constants 5
2.4 – System Block Diagram 7
2.5 – System Functional Description 7
3 – Mechanical Systems 9
3.1 – Component Assembly Layout 9
3.2 – Component Descriptions 9
4 – Electrical Systems 10
4.1 – Overview of Electrical Subsystems 10
4.2 – Specifications for Electrical Components 11
4.3 – Electrical Wiring and Pinout/Port Diagram 12
5 – Software | General 13
5.1 – General Software Structure 13
5.2 – Initialization Procedure 14
5.3 – Initialization and IDLE Algorithm 15
5.4 – Initialization/IDLE/Housekeeping Functions 16
5.5 – PWM Generation 17
6 – Software | Motor 19
6.1 – Motor Behaviour 19
6.2 – Motor Functions 19
7 – Software | Data Collection and Transmission 21
7.1 – Data Collection and Transmission Algorithm 21

7.2 – Data Transmission Pathway 25
7.3 – Data Collection, Processing, and Transmission Functions 25
7.4 – Error Code Interpretation and Correction 27
8 – Software | Data Visualization 28
8.1 – Data Receiving and Visualization Program Structure 28
8.2 – Data Storage 28
8.3 – Data Receiving Algorithm 28
8.4 – Data Processing and Visualization Algorithm 29
9 – Application Procedure and Limitations 30
9.1 – Application Procedure: 30
9.2 – Application Example: 30
10 – Device Limitations: 32
10.1 – Floating Point Errors and Trigonometric Calculations 32
10.2 – Maximum Baud Rate of PC 32
10.3 – Communication Speeds of UART and I2C 33
10.4 – Speed Bottleneck of ESM System 33
11 – References and Additional Reading 34
11.1 – Additional Reading 34
11.2 – References 34

1 – Device Overview:

1.1 – General Description:

The Embedded Spatial Measurement (ESM) system is a Light Detection and Ranging (LIDAR) embedded system used to map out 3D geometry by scanning and plotting 2D slices of terrain via distance measurements relative to a central axis of motion.



The ESM system can measure distances in a 360° radius, up to 3.6m away, using a Time of Flight Sensor. The ESM features:

- Rapid scanning capabilities (50-200ms between measurements)
- Configurable accuracy:
    - Default per-scan accuracy is 5 points
    - Default per-slice accuracy is 32 scans
- Automatic error detection
- Ergonomic chassis
- LED-based troubleshooting system
- Asynchronous storage and plotting

The ESM is housed in a 3D printed chassis designed for maximum compatibility with ESM-specific systems and can be assembled via friction-fitting.

1.2 – Purposes:

The ESM system is used for spatial mapping and processing purposes, including:

- 2D/3D Mapping
- Ranging/Distance Measurements
- Object Detection

The ESM system is best used in average-light conditions and in relatively closed spaces with non-reflective surfaces.

1.3 – Terminology:

Some technical terms to improve legibility and consistency are stated below. Please refer to these terms for convenience.

Data Set	A group of measurements used to form a scan
ESM	Embedded Spatial Measurement
MSP-401Y, MC	MSP-EXP432E401-Y microcontroller
PC	Personal computer
Scan	A single point in a slice
Slice	A 2D scan of a larger 3D volume created in two dimensions.
Step	The minimum angle increment of the motor
ToF	Time of Flight sensor

2 – Product Overview

2.1 – Technical Specifications

Feature	*Detail*
Bus Speed	20 MHz using Phase-Locked Loop
Communication Protocol Speed	UART: 112500 baud rate I2C: 100 kb/s (Standard Mode)
Cost	$250 for raw electrical components, sourced from McMaster 2DX3 components kit 3D Printed chassis is free
Hardware/Software Requirements	Hardware: • Personal computer with Micro USB-A adaptor port Software: • MATLAB • UART Functionality
Memory Requirements	Default MC capacity is 32 readings per slice. MATLAB capacity is system dependent. • Recommended no more than 1 Mb of storage (plenty!)
Operating Voltage	5V – Micro A USB Cable
Package	3D Printed – 4 Components, Friction-Fitted • Microcontroller Chassis • Motor Arm Mount (x2) • ToF Mount Base • ToF Mount Cage
Range	Programmable, automatically changes range modes. • Short Mode (Default) – 1.3m in ambient lighting conditions • Long Mode – Up to 3.6m in dark lighting conditions
Scan Speed	32 scans per slice – 5 min/slice 64 scans per slice – 10 min/slice
Size	185 x 173 x 41.5mm
Software Languages	C, MATLAB

2.2 – System Operating Modes

Mode	*Behaviour*
IDLE	ESM waits for new instructions and does not conduct any operation. Default mode.
SCAN	ESM is generating a new slice by scanning surroundings with ToF sensor.
TRANSMIT	ESM is processing raw slice data and transmitting data to the PC.

2.3 – System Settings and Constants

Certain aspects of the ESM can be configured with simple code/hardware modifications. Other aspects remain constant for reference and are more difficult to change.

Constants:

Constant	*Description*	*Value*
Baud Rate	Rate of bit transmission	115200
Bus Speed	Operating speed of MSP-401Y microcontroller	20MHz
I2C Communication Rate	Speed of I2C bit transmission	100kb/s
LED Blink Rate	Speed of LED Blinking	**Fast:** 100ms period **Slow:** 50ms period
Slice Plane	Cartesian plane in which one slice is completed	XY Plane
ToF Follower Address	I2C Follower Address of ToF sensor	0x29

Settings:

Setting	*Description*	*Default Value*
Angle Step	Stepper motor angle between measurements Calculated via per-slice accuracy. See below.	11.25°
Attempts Per Measurement	Number of attempts made to obtain a measurement Modify in Keil µVision • Keil → NewD2.c → Line 41 ○ Modify constant	5
Inter-Measurement Period	Minimum time between measurements Modify in Keil µVision • Keil → NewD2.c → Lines 313-323 → *VL53L1X_SetInterMeasurementInMs(dev,* ***Target)*** ○ Modify constant	**Short Mode:** 100ms **Long Mode:** 200ms

Per-Scan Accuracy	Number of measurements made per scan.
	
	Modify in Keil µVision and MATLAB
	• Keil → NewD2.c → Line 39
	  ○ Modify constant
	• Keil → NewD2.c → Line 48
	  ○ Modify array length
	• MATLAB → readData.m → Line 10
	  ○ Modify constant
	• MATLAB → visualizeData.m → Line 9
	  ○ Modify constant
		5

Per-Slice Accuracy	Number of scans made per slice.
	
	Modify in Keil µVision
	• Keil → NewD2.c → Line 40
	  ○ Modify constant
	• Keil → NewD2.c → Line 50
	  ○ Modify array length
	• Keil → NewD2.c → Line 53
	  ○ Modify array length
		32

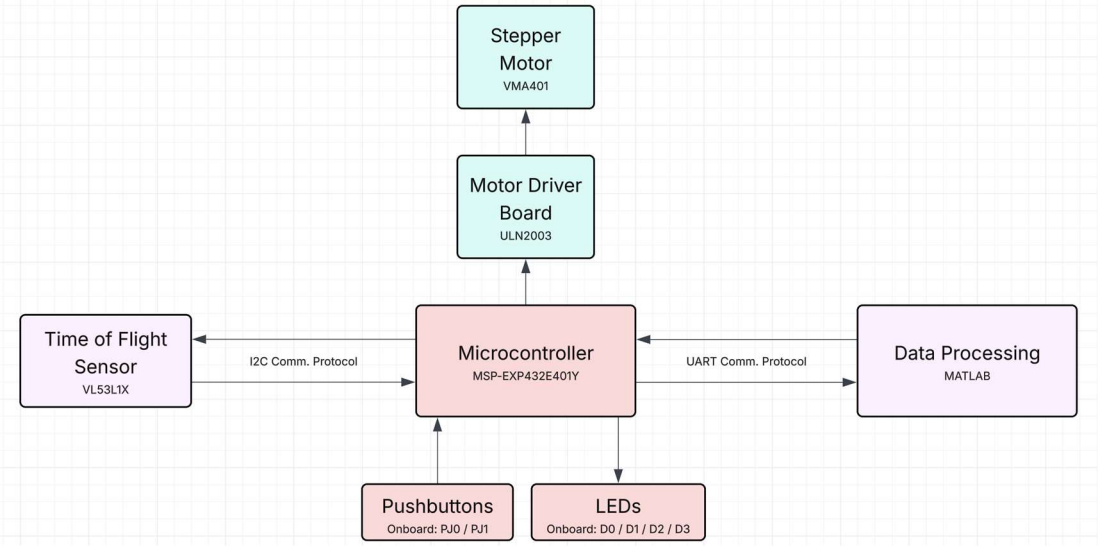
Port	UART Port used for data communication between MC and personal computer Modify in MATLAB • MATLAB → readData.m → Line 8 ○ Modify constant	COM5
Timing Budget	Time required, in milliseconds, to complete one measurement. Modify in Keil µVision • Keil → NewD2.c → Lines 313-323 → *VL53L1X_SetTimingBudgetInMs(dev,* ***Target****)* ○ Modify constant	**Short Mode:** 50ms **Long Mode:** 100ms
UART Timeout Period	Amount of time to keep UART port open before automatically closing Modify in MATLAB • MATLAB → readData.m → Line 15 ○ Modify constant	3600s

Z-Dimension Distance	Assumed distance in Z direction made between scans Modify in MATLAB • MATLAB → readData.m → Line 11 ○ Modify constant	300mm

2.4 – System Block Diagram

The ESM system is divided into subsections

- Red = Control System
- Pink = Data Transmission System
- Green = Motor System



2.5 – System Functional Description

The user can control the ESM system via onboard pushbuttons on the MSP microcontroller:

Pushbutton	*Function*
RESET	Restarts ESM to IDLE. Cancels any currently active operation and wipes all data stored on the microcontroller.
PJ0	Toggles SCAN mode. Starts a new scan when a scan is not active. • Defines current ToF position as the positive x-axis Stops the current scan if one is in progress.

	• Keeps current motor position with respect to the positive x-axis.
PJ1	Activates TRANSMIT mode. Stops the current scan and initializes data transfer to the personal computer. Transmit mode must be completed before new data slice can be initialized.

In SCAN mode:

1. The ToF sensor measures the radial distance repeatedly.
    a. Number of measurements depends on the per-scan accuracy setting.
2. Motor steps counterclockwise by the required angle.
    a. Angle is determined by the per-slice accuracy.
3. Steps 1 and 2 are repeated until a full slice is produced.
    a. Number of repetitions is determined by the per-slice accuracy.

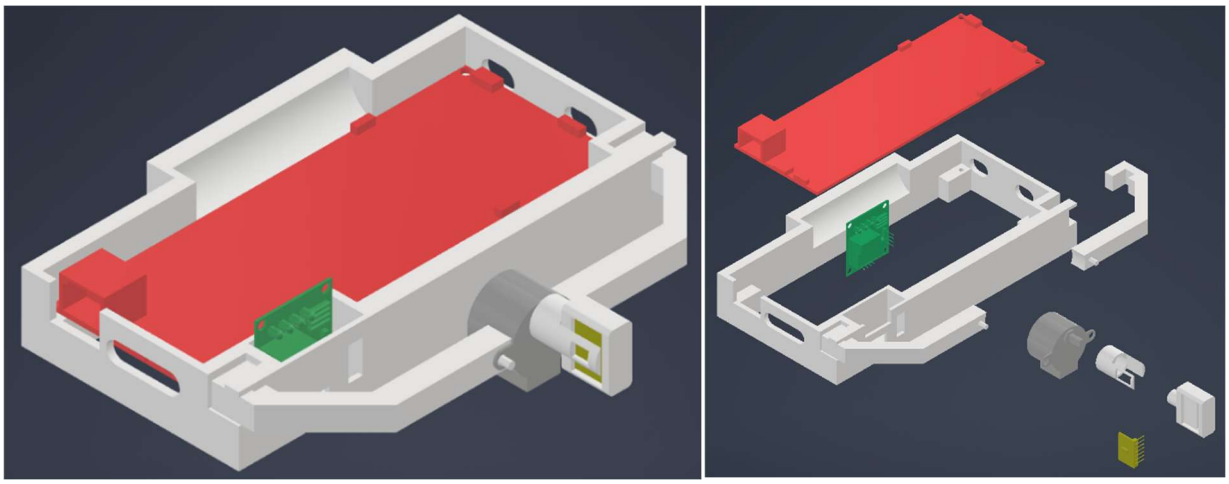
In TRANSMIT mode:

1. Device waits until the start flag is received.
    a. The start flag is transmitted by MATLAB.
2. Upon receiving the start flag, the device transmits data to MATLAB.
3. Device returns to IDLE mode.

3 – Mechanical Systems

3.1 – Component Assembly Layout

Note: all colored components are models of electrical components, and are only present for reference, not to be printed.



3.2 – Component Descriptions

All components can be printed with a standard PLA 3D printer. All non-friction-fit intersections are given a minimum tolerance of 2mm. All prints are completed with maximum supports.

Component	*Description*	*Printing Time*
Microcontroller Chassis	Holds the MSP-401Y and ULN2003 motor board.	4h56m
Motor Arm Mount	Holds the VMA401 stepper motor. Requires two copies.	26m
ToF Mount Base	Holds the VL53L1X ToF sensor to shaft of motor.	17m
ToF Mount Cage	Additional shielding for ToF sensor. Optional.	17m

4 – Electrical Systems

4.1 – Overview of Electrical Subsystems

There are four defined electrical subsystems that serve specific purposes in the ESM system.

Data Transduction and Transmission System

Readings are first generated from the ToF sensor, which are then processed by the microcontroller and the PC through I2C and UART lines, respectively.

LED Troubleshooting System

Onboard LEDs flash to signify different stages/occurrences in the scanning and transmitting process.

D1 – Measurement indicator

- Blinks for each measurement made by ToF sensor

D2 – UART transmission indicator

- Blinks when UART transmission is first initialized

D3 – Error measurement indicator

- Turns on when ToF sensor makes an erroneous measurement
- Turns off when ToF sensor makes a valid measurement

D4 – Slice capacity indicator

- Turns off when data slice array is empty
- Blinks slowly when data slice array is less than half-full
- Blinks rapidly when data slice array is over half-full
- Turns on when data slice array is at capacity

All LEDs Flash – ToF initialization indicator

- All LEDs flash once when ToF finishes initialization

Motor Control System

DC stepper motor is controlled by a series of four stator coils, each managed by a GPIO pin. Allows for precise control by controlling the data registers of corresponding pins.

Pushbutton Control System

Pushbuttons are configured as interrupt-based inputs to control the operating mode of the ESM system. *(See Section 2.5)*

4.2 – Specifications for Electrical Components

Microcontroller – MSP EXP432E401-Y

Specification	*Description*
Bus Frequency	20MHz
Onboard LEDs	Four onboard LEDs active (D1/D2/D3/D4)
Onboard Pushbuttons	Two onboard pushbuttons active (PJ0/PJ1)
Operating Voltage	5V supplied by PC

Time of Flight Sensor – VL53L1X

Specification	*Description*
Operating Voltage	3.3V supplied by MC
Method of Operation	Near-infrared light (940nm) – LIDAR Light is emitted from the transmitter and the time required for the receiver to detect the emitted wavelength is used to calculate distance according to the formula: $$d = \frac{c \cdot t}{2}$$
Pinout	Four pins required • V_{IN} = Input voltage • GND = Ground connection (common with MC) • SDA = Data transmit/receive line • SCL = ToF clock line

Motor and Driver Board – VMA401/ULN2003

Specification	*Description*
Operating Voltage	5-12V supplied by MC
Method of Operation	For any motor step, two specific stators are powered on, while two stators are powered off, so the motor can rotate appropriately.
Pinout	Six pins required

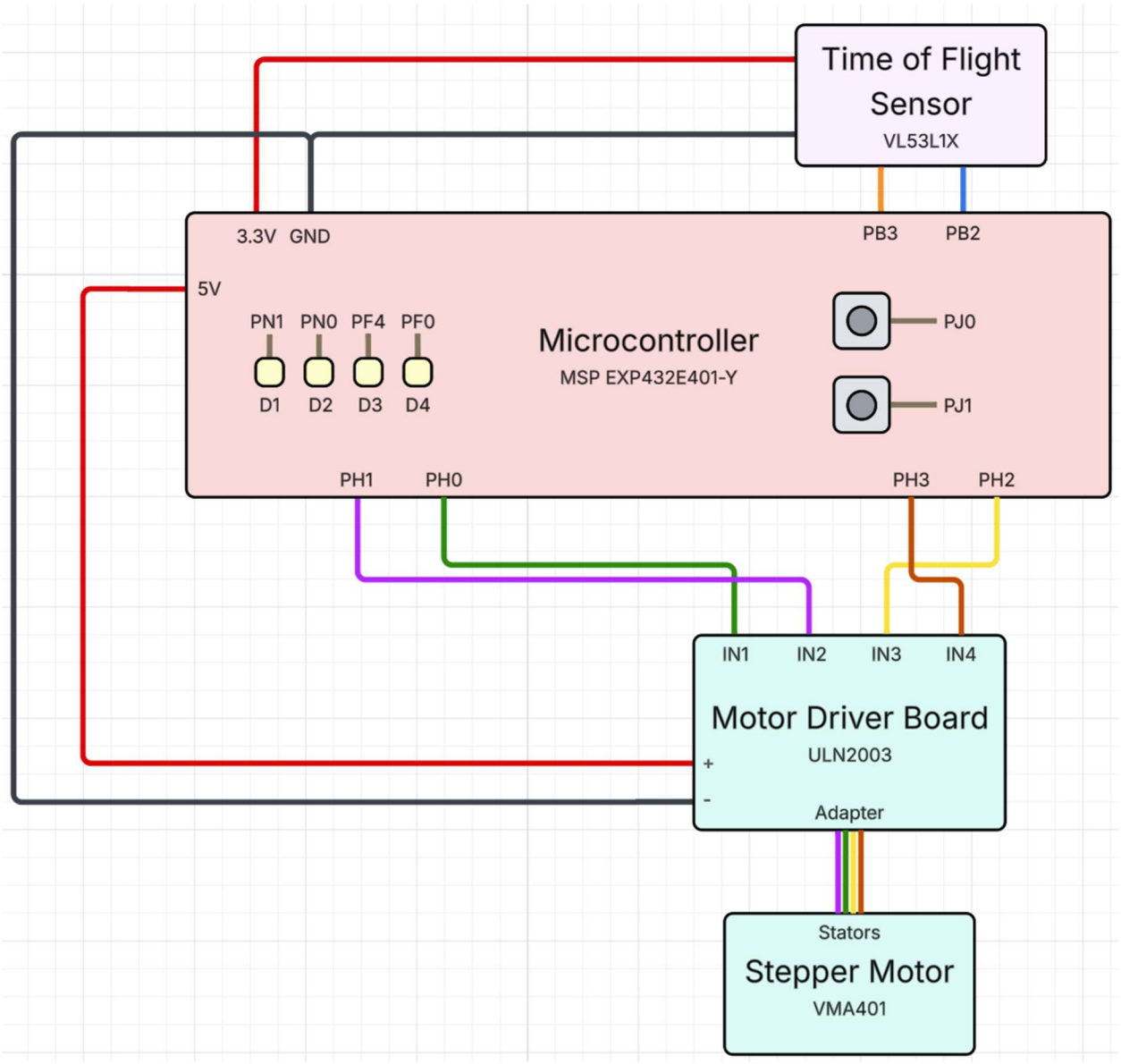
	• V_{IN} = Input voltage • GND = Ground connection (common with MC) • IN1/IN2/IN3/IN4 = Control pins for motor stators

4.3 – Electrical Wiring and Pinout/Port Diagram

Various independent subsystems that are wired to the central microcontroller. By default, specific pins are initialized and assigned to various systems

MSP-401Y Pin	*Peripheral Component*	*Configuration*	*Purpose*	*Description*
PB2	ToF Sensor	Transmission	Clock Line	Sends clock signals to synchronize ToF data transmission with MC
PB3	ToF Sensor	Transmission	Data Line	Transmits data bits between ToF and MC
PH0	Stepper Motor	OUT	Stator IN1 Control	Digital control for stator 1
PH1	Stepper Motor	OUT	Stator IN2 Control	Digital control for stator 2
PH2	Stepper Motor	OUT	Stator IN3 Control	Digital control for stator 3
PH3	Stepper Motor	OUT	Stator IN4 Control	Digital control for stator 4
PN1	Onboard LED	OUT	LED D1 Control	Onboard LED for measurement indication
PN0	Onboard LED	OUT	LED D2 Control	Onboard LED for UART indication
PF4	Onboard LED	OUT	LED D3 Control	Onboard LED for error indication
PF0	Onboard LED	OUT	LED D4 Control	Onboard LED for storage capacity indication
PJ0	Onboard Pushbutton	IN	Pushbutton PJ0 Control	Onboard pushbutton for scanning
PJ1	Onboard Pushbutton	IN	Pushbutton PJ1 Control	Onboard pushbutton for transmission
PM0	GPIO Register	OUT	PWM Generation	Generates a 50% duty cycle PWM wave of a given frequency for clock speed testing.

A pinout diagram is displayed below.



5 – Software | General

5.1 – General Software Structure

The ESM system has three general algorithms that can be called to complete its three primary functions (IDLE/SCAN/TRANSMIT). The ESM system is interrupt-based, with both input pushbuttons toggling flags to activate/deactivate functions. The main body of the code will repeatedly check if the flags have been toggled and will execute functions based on the interrupt state. Furthermore, in IDLE mode, the storage state LED will repeatedly flash to indicate the storage capacity of the slice.

5.2 – Initialization Procedure

The ESM system begins with an initialization period to properly initialize all required libraries, ports, and systems.

Port F

Specification	*Description*
Pin Enable	PF0, PF4
Direction	PF0 → Output PF4 → Output
Data (Initial)	PF0 → 0 PF4 → 0

Port H

Specification	*Description*
Pin Enable	PH0, PH1, PH2, PH3
Direction	PH0 → Output PH1 → Output PH2 → Output PH3 → Output
Data (Initial)	PH0 → 0 PH1 → 0 PH2 → 0 PH3 → 0
Analog/Alternate Functionality	PH0 → Disabled PH1 → Disabled PH2 → Disabled PH3 → Disabled

Port J

Specification	*Description*
Pin Enable	PJ0, PJ1
Direction	PJ0 → Input PJ1 → Input
Data (Initial)	PJ0 → 0 PJ1 → 0
Interrupt Sensitivity	Edge sensitive
Single/Double Edge	Single edge only
Rising/Falling Edge	Falling edge only
Interrupt Pin Enable	PJ0, PJ1
Interrupt Clear Register	PJ0, PJ1
NVIC IRQ Number	IRQ number is 51

Priority Level	Level 3 priority (out of 7)

Port M

Specification	*Description*
Pin Enable	PM0
Direction	PM0 → Output
Data (Initial)	PM0 → 0

Port N

Specification	*Description*
Pin Enable	PN0, PN1
Direction	PN0 → Output PN1 → Output
Data (Initial)	PN0 → 0 PN1 → 0

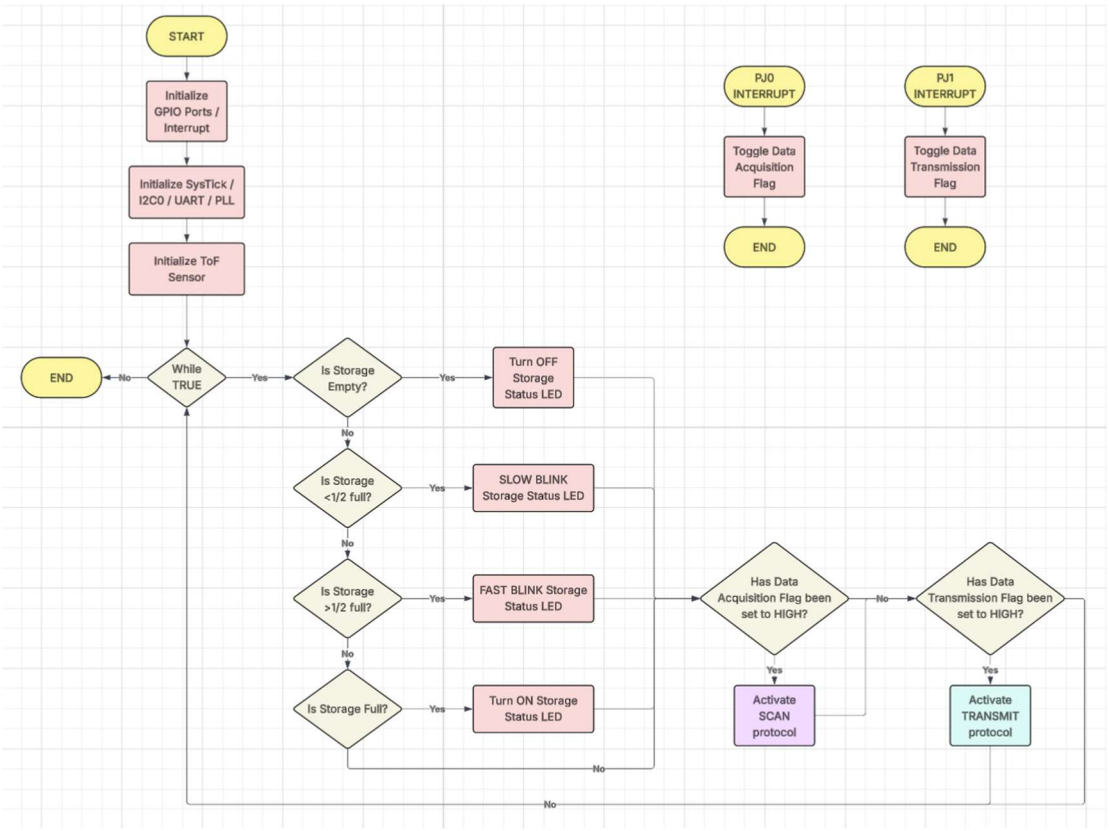
Additionally, certain external libraries and components must be initialized by calling their own specialized initialization functions. These include the SysTick, VL53L1X API, UART, I2C0, and PLL libraries.

- SysTick → Enables delay functionality based on system clock frequency
- VL53L1X API → Facilitates basic processes with ToF sensor
- UART → Facilitates UART communication
- I2C0 → Facilitates I2C communication
- PLL → Enables Phase-Locked Loop for custom clock signal (20 MHz)

Finally, a custom initialization function is created to initialize the ToF sensor. More details can be found in *Section 5.4*.

5.3 – Initialization and IDLE Algorithm

When first booted, the ESM system will initialize all required libraries and GPIO ports, before entering a **while True** loop in the main body of code. The IDLE algorithm is activated on startup/when another function has completed.



5.4 – Initialization/IDLE/Housekeeping Functions

ToF_Init

Input(s)	*Output(s)*	*Lines*	*Description*
VOID	VOID	325 - 338	Initializes ToF sensor in preparation to send/receive data. 1. Wait for BootState Function to confirm ToF has fully booted 2. Initialize sensor using *ToF_SensorInit();* 3. Set initial ToF ranging mode to short mode

ASSORTED LED ON/OFF/BLINK

Input(s)	*Output(s)*	*Lines*	*Description*
VOID	VOID	144 - 255	Assorted functions to turn LEDs ON/OFF by controlling GPIO data register associated with LED pin. Assorted functions to make LEDs blink • Fast = 50ms period

			• Slow = 100ms period (default)

GPIOJ_IRQHandler

Input(s)	*Output(s)*	*Lines*	*Description*
VOID	VOID	233 - 242	Toggles a flag to control operating mode depending on which button is pressed. PJ0 → Toggle *dataAcquisitonFlag* to control SCAN mode. PJ1 → toggle *dataTransmissionFlag* to control TRANSMIT mode.

generateWave

Input(s)	*Output(s)*	*Lines*	*Description*
VOID	VOID	249 - 257	Generates a 50% square wave with a roughly 50Hz frequency. Used for clock speed troubleshooting

main

Input(s)	*Output(s)*	*Lines*	*Description*
VOID	INT	544 - 591	Main code loop: Initializes all necessary libraries and components by calling respective initialization functions. Repeatedly check if *dataAcquisitionFlag* is toggled, initializes scan*On()* if turned on. • Activates SCAN mode. Repeatedly check if *dataTransmissionFlag* is toggled, initializes *convertToString()* and *UARTTransmission()* if turned on. • Activates TRANSMIT mode. Flashes storage status LED based on capacity.

5.5 – PWM Generation

An additional PWM generator is available for troubleshooting clock speed. When activated, a 50% duty cycle PWM wave is produced at PM0, and its frequency can be measured with an oscilloscope. If the frequency of the wave is 50Hz, the microcontroller must be operating at 20MHz.

Assuming our clock speed is 20MHz, the period of one clock cycle must be:

$$Period = \frac{1}{20\ MHz} = 50ns$$

We generate the PWM signal using *SysTick_Wait*, which delays by a specific number of clock cycles. To obtain a frequency of 50Hz, (i.e. a period of 0.02s):

$$Number\ of\ Delays = \frac{0.02s}{50ns} = 400000$$

Split across two delays, each *SysTick_Wait* function will delay for 200000 clock cycles. Thus, to confirm the clock speed, measure the frequency of the PWM generator and compare to the ideal 50Hz signal.

6 – Software | Motor

6.1 – Motor Behaviour

The VMA401/ULN2003 stepper motor and driver board uses four stators that can be controlled with digital signals from GPIO output pins. When two consecutive pins are raised HIGH, the stators activate, moving the rotor. Rotation is achieved by timing the activation of subsequent stators.

$$0011 \rightarrow 0110 \rightarrow 1100 \rightarrow 1001$$

The above diagram demonstrates the action of stators that produces *one step* of motor rotation, in the order of IN1 → IN2 → IN3 → IN4

Property	*Description*	*Value*
Steps per Rotation	Number of steps to complete one 360° rotation.	512 steps
Degrees per Step	Number of degrees per step of rotation	0.703125°
Steps Between Scans	Number of steps between subsequent scans. $Number\ of\ Instances = \frac{512}{READINGSPERSCAN}$	Calculated

6.2 – Motor Functions

motorClockwiseStep* | *motorCounterClockwiseStep

Input(s)	*Output(s)*	*Lines*	*Description*
VOID	VOID	449 - 471	Rotates motor by the smallest possible step in the clockwise or counterclockwise direction.

blinkClockwiseStep* | *blinkCounterClockwiseStep

Input(s)	*Output(s)*	*Lines*	*Description*
VOID	VOID	473 - 487	Rotates motor by required number of degrees to obtain proper number of readings per scan. Moves clockwise or counterclockwise.

resetHome

Input(s)	*Output(s)*	*Lines*	*Description*

VOID	VOID	489 - 505	Resets motor position by rotating clockwise for the appropriate number of steps. Resets appropriate variables to default position • dataAcquisitionFlag • dataTransmissionFlag • readingsCompleted • readingsString Resets ToF distance mode to short mode Clears arrays for storing readings per scan and scans per slice.

7 – Software | Data Collection and Transmission

7.1 – Data Collection and Transmission Algorithm

Data collection and transmission occur in the following pathway in Keil:

1. MC begins measurement for new reading.
2. ToF sensor informs MC via I2C pathway that a new reading has been generated.
3. ToF sensor transmits reading to MC via I2C pathway. The following information is transmitted:

Data Point	*Description*	*Valid Range*
distance	Obtains the distance in mm. Range is most accurate above 40mm.	0 – 3600mm
distanceMode	Obtains the ToF's current distance mode.	1 – Short 2 – Long
rangeStatus	Obtains the error code associated with a measurement	0 – Valid 1 – Repeatability Error 2 – Weak Signal 4 – Out of Range 7 – High Reflectivity

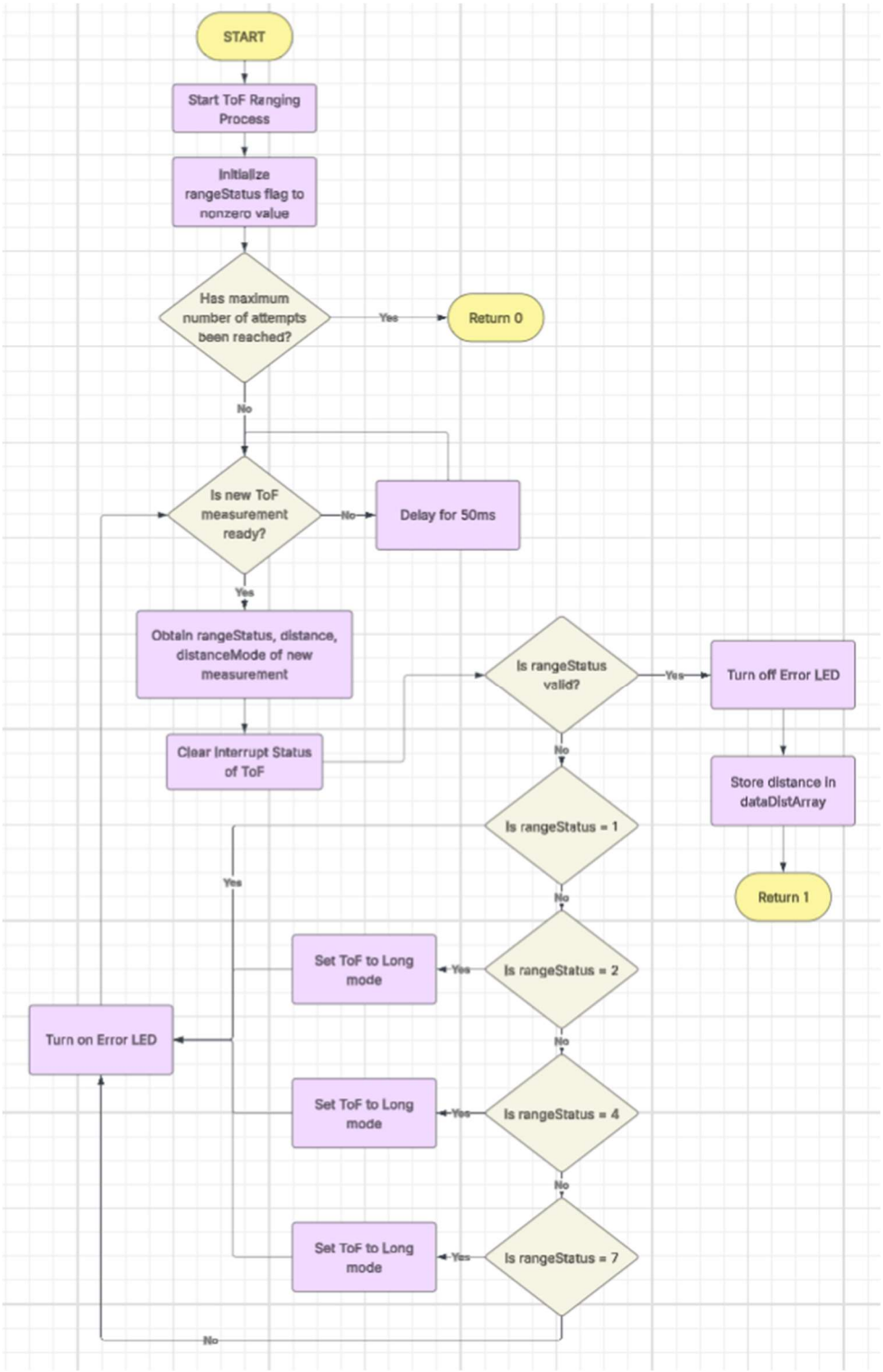
4. MC accepts the distance measurement if no error has occurred and retakes the measurement if an error has occurred and attempts are still permitted.
    a. The maximum number of attempts is configurable *(See Section 2.3)*
    b. If accepted, the measurement is stored in an array as part of a data set (*dataDistArray*).
5. MC repeats measurements until data set array is filled.
6. MC calculates the average measurement from the array and stores the result in an array (*readings*) to hold the measurements in a slice.
7. MC scans repeatedly until slice array is filled.
8. Slice array is converted to string (*readingsString*)
    a. A delimiter "/" is added to distinguish subsequent scans.
    b. Ex: The below string is 4 scans of length 145mm, 3974mm, 2943mm, and 47mm.

0145/3974/2943/0047

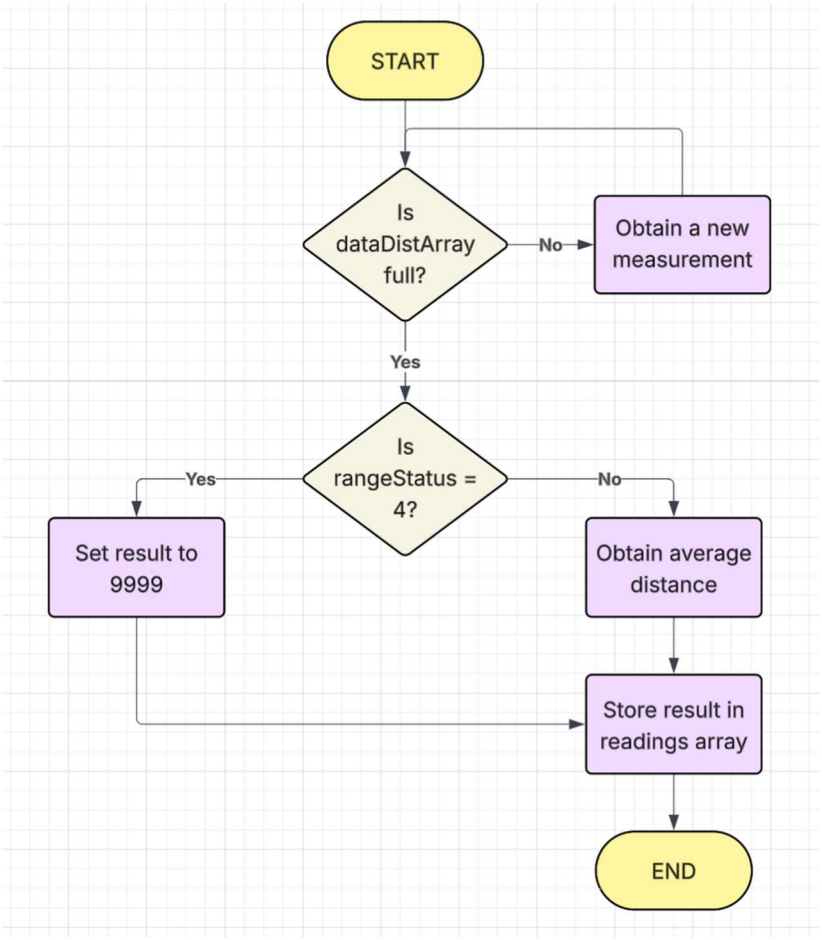
9. MC waits for start flag from PC for UART communication
10. PC transmits start flag to MC
11. MC transmits data string to PC

Flowcharts below describe the SCAN and TRANSMIT algorithms.

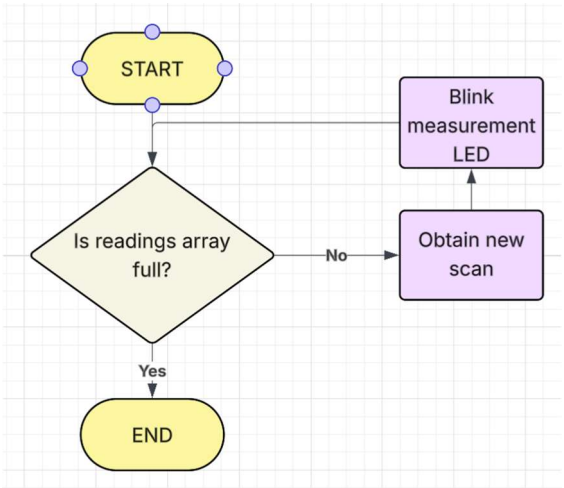
SCAN – Per Measurement:



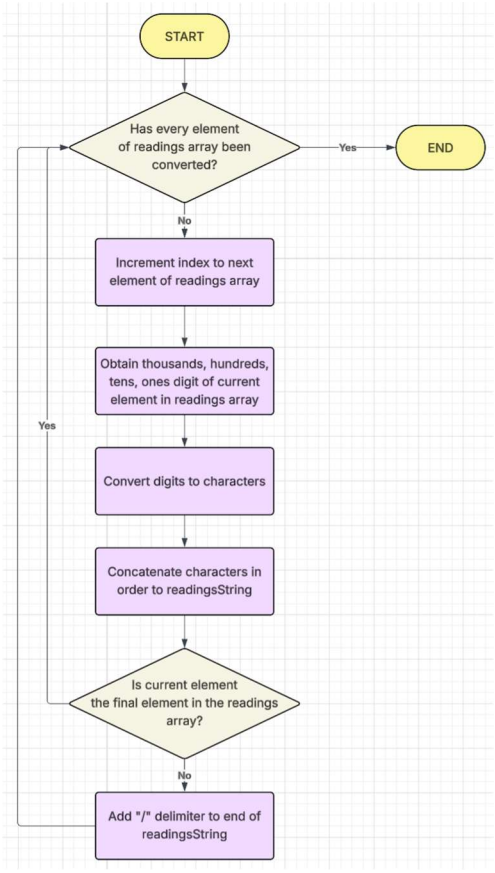
SCAN – Per Scan:



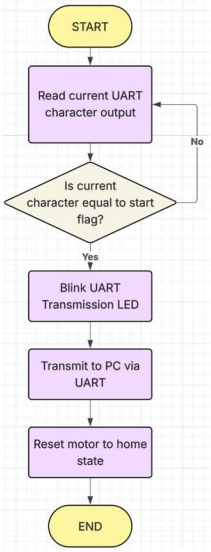
SCAN – Per Slice:



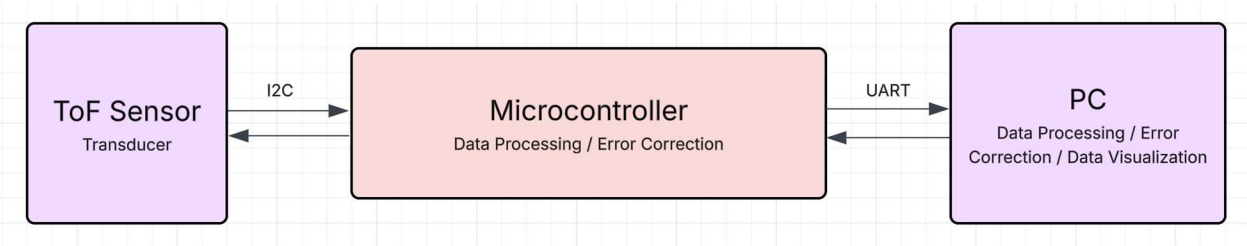
TRANSMIT – Data Conversion to String:



TRANSMIT – Transmission to PC:



7.2 – Data Transmission Pathway



Note the speeds of data transmission differ in I2C and UART.

Communication Protocol	*Speed*
UART	112500 bits/s
I2C	100 kb/s

7.3 – Data Collection, Processing, and Transmission Functions

clearReadings

Input(s)	*Output(s)*	*Lines*	*Description*
VOID	VOID	265 - 270	Clears *readings* array

clearDataDistUnit

Input(s)	*Output(s)*	*Lines*	*Description*
VOID	VOID	272 - 277	Clears *dataDistUnit* array

convertToString

Input(s)	*Output(s)*	*Lines*	*Description*
VOID	VOID	279 - 305	Converts entries in *readings* array from uint16 to characters and appends the characters to a string. For each entry, appends the four digits of the entry and a “/” delimiter.

scanOn

Input(s)	*Output(s)*	*Lines*	*Description*
VOID	VOID	507 - 516	Main function for executing SCAN operating mode Repeatedly calls *ToF_collectDataUnit* and *blinkCounterClockwiseStep* until *readings* array is full

ToF_validateOne

Input(s)	*Output(s)*	*Lines*	*Description*
rangeStatus	INT	340 - 377	Validates the current measurement based on *rangeStatus*. Outputs success/failure code 1 – Valid result 0 – Invalid result *(See Section 7.4)*

ToF_measureOne

Input(s)	*Output(s)*	*Lines*	*Description*
index	INT	379 - 413	Obtains one measurement, validates it, and stores it in *dataDistUnit* Outputs success/failure code 1 – Measurement successful 0 – Measurement failed

ToF_collectDataUnit

Input(s)	*Output(s)*	*Lines*	*Description*
i	VOID	415 - 441	Obtains one data set, validates it, and stores it in *readings* • If ToF_measureOne fails to return a measurement, sets data unit to 9999. o This is an impossible measurement that can be filtered in MATLAB

UARTTransmission

Input(s)	*Output(s)*	*Lines*	*Description*
VOID	VOID	524 - 537	Waits for data transmission start flag, then transmits *readingsString* to PC via. UART line.

			Blinks UART transmission LED. Calls resetHome() to reset motor position and variables.

7.4 – Error Code Interpretation and Correction

The ToF sensor has five possible error codes that are used to determine corrective measures.

Error Code	*Description*	*Corrective Action*
0	No error detected. Measurement valid.	None.
1	Repeatability Error. Measurement is inconsistent with high standard deviation.	Repeat measurement.
2	Weak Signal	Change ToF distance mode to Long. Repeat measurement.
4	Out of Range	Change ToF distance mode to Long. Repeat measurement.
7	High Reflectivity	Change ToF distance mode to Long. Repeat measurement.

Furthermore, if the maximum number of attempts to make a measurement is reached, the scan distance is automatically converted to 9999. This is an impossible measurement that can be filtered out in MATLAB.

8 – Software | Data Visualization

8.1 – Data Receiving and Visualization Program Structure

Data receiving and visualization is completed in MATLAB. The visualization program is composed of four MATLAB scripts:

Script	*Description*
newMeasureData.m	Wipes MatlabMeasureData.mat
readData.m	Receives the raw scan data from MC. And stores in MatlabMeasureData.mat
MatlabMeasureData.mat	Stores the raw scan data from MC.
visualizeData.m	Processes raw scan data from MatlabMeasureData.mat

8.2 – Data Storage

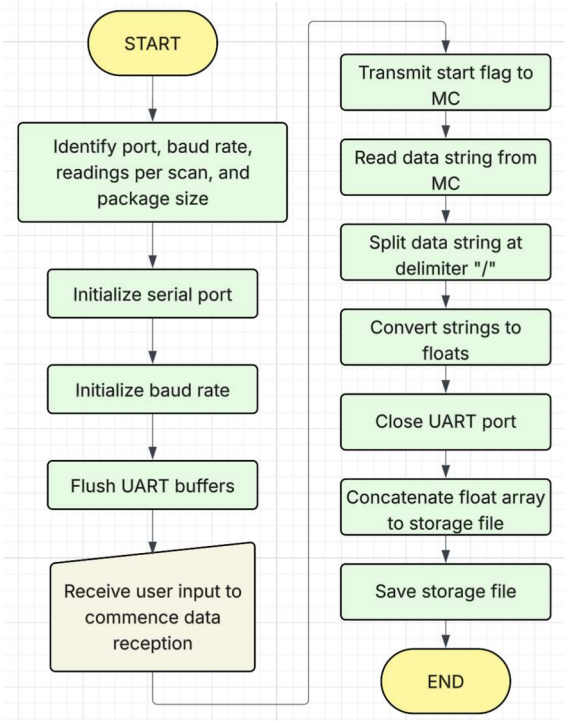
Data is transferred to PC by slice. Upon receiving, the data is transferred to a .mat file *MatlabMeasureData.mat* for storage.

- The new slice is concatenated on top of the current data. New data slices do NOT have to be created in one action, if the most recent scan has been uploaded to the MATLAB storage.

To produce a new graph, ensure that *MatlabMeasureData.mat* has been cleared using *newMeasureData.m*.

8.3 – Data Receiving Algorithm

MATLAB will receive and store the data slice transferred by the MC. Below is a flowchart describing the process.



8.4 – Data Processing and Visualization Algorithm

Raw data needs to be processed with a simple algorithm to obtain X and Y coordinate lengths from the radial distance. This is done with simple trigonometry. Z-distance is constant for every distinct slice and increments by a constant amount for subsequent slices.

Value	*Process*
x-coordinate distance	$xDist = radialDist \cdot \cos(angleIncrement \cdot (scanIndex - 1))$
y-coordinate distance	$yDist = radialDist \cdot \sin(angleIncrement \cdot (scanIndex - 1))$
z-coordinate distance	$zDist = \text{sliceIndex} \cdot \text{zIncrement}$

Note that the angleIncrement and zIncrement are set as constants in the MATLAB script.

Finally, the coordinates are plotted on a 3D plot and a surface plot is generated.

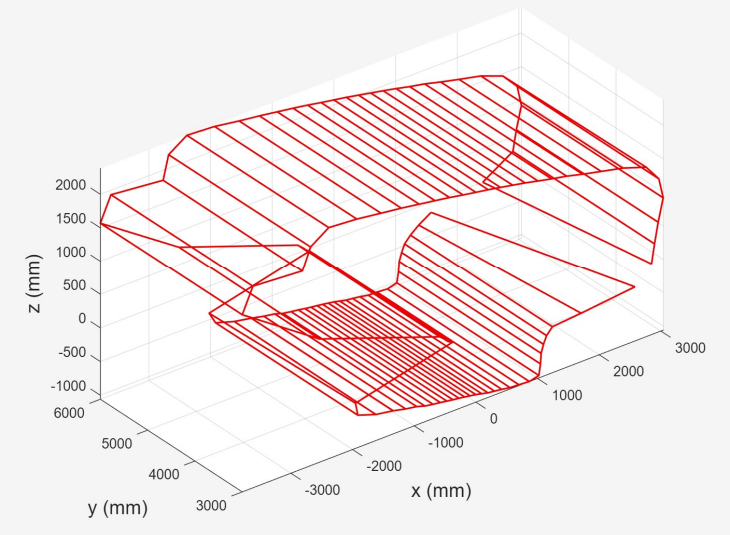
9 – Application Procedure and Limitations

9.1 – Application Procedure:

To use the ESM system:

1. Identify the scan location desired.
    a. Recommended locations include semi-enclosed areas with minimal reflective surfaces and dark-to-ambient lighting.
2. Configure all settings as desired.
    a. *See Section 2.3.*
3. Wipe the memory of the MATLAB script using *newMeasureData.m.*
4. Power on the ESM device by connecting the MC to the PC using a Micro USB-A connector.
5. Wait for all LEDs to blink.
    a. This signifies that the ToF has initialized properly.
6. Begin the first slice by pressing PJ0 to activate SCAN mode.
7. Allow the slice to complete.
    a. Recommendations:
        i. Do not move the ESM device while the scan is active for ideal results.
        ii. Do not allow obstructions (wires, particles, etc.) in front of the ToF sensor for ideal results.
    b. When slice is completed, LED D4 will be on to signify the array is at capacity.
8. Press PJ1 to activate TRANSMIT mode.
9. On the PC, run *readData.m* and press ENTER when prepared to receive data from MC.
10. Repeat steps 6 – 9 as desired.
    a. Increased slices results in increased accuracy.
11. Run *visualizeData.m* to generate a graph.

9.2 – Application Example:



A sample measurement is provided above of a scan of a bedroom with the following settings:

Setting	*Value*
Angle Step	5.625º
Attempts Per Measurement	5
Inter-Measurement Period	**Short Mode:** 100ms **Long Mode:** 200ms
Per-Scan Accuracy	5
Per-Slice Accuracy	64
Port	COM5
Number of Slices	2
Timing Budget	**Short Mode:** 50ms **Long Mode:** 100ms
UART Timeout Period	3600s
Z-Dimension Distance	3000mm

Note that scans are completed in the XY plane and the Z plane is the axis of forward movement.

10 – Device Limitations:

10.1 – Floating Point Errors and Trigonometric Calculations

MATLAB has finite floating-point unit (FPU) precision. This means there is an inherent limitation to the number of decimal places that MATLAB can handle. The maximum float value that MATLAB can handle is 1.7977E+308 in a double data format, which is the default float data type [1].

Furthermore, trigonometric functions often evaluate to irrational numbers, meaning they have no limit to their size and are considered to have infinite precision. Thus, MATLAB is forced to round irrational values of trigonometric functions, resulting in floating point errors. In practice, the ESM system is largely unaffected by floating point errors due to other, more significant sources of error, such as environmental noise, which have a significantly greater impact.

10.2 – Maximum Baud Rate of PC

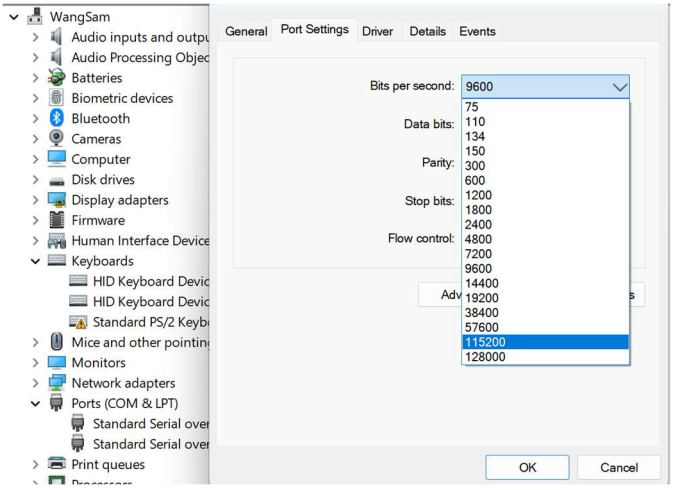
To find the maximum baud rate of the computer, we can use the *Device Manager*.

Device Manager → Ports (COM & LPT) → <Selected COM Port>

Right click on the selected port and click:

Properties → Port Settings → Bits per Second

Select the dropdown. The largest value is the maximum baud rate supported by the computer.



For example, the current hardware supports a baud rate of up to 128000 bits per second.

10.3 – Communication Speeds of UART and I2C

In the ESM system, the I2C communication speed is set to *Standard* mode, at 100 kb/s. Furthermore, the UART communication speed is set to 112500 bits/s, or 112.5 kb/s. If UART and I2C were transmitting simultaneously and continuously, it is possible for data to become desynchronized as data is transmitted. However, the ESM system does not transmit data continuously, so this is not an issue.

10.4 – Speed Bottleneck of ESM System

Overall, the primary speed bottleneck of the ESM system is the number of measurements made by the ToF sensor. Each measurement must have a minimum inter-measurement delay of 200 ms in long mode (dominant), which results in each scan (five measurements per scan) taking at least 1 second, excluding retakes from error measurements. Over 32 scans, each slice will take a over of 32 seconds to complete from ToF measurements alone. This bottleneck was confirmed by code analysis to count the number of delays experienced in each measurement.

Another significant bottleneck is the delay requirement of the stepper motor. A 20-millisecond delay is allotted to each motor step. Since 512 steps are required for one full slice, this results in a delay of 10.24 seconds per full cycle. This bottleneck was confirmed by code analysis to count the number of delays experienced in one motor step.

A less significant but still potential bottleneck is with ToF initialization and dataReady. Both actions require the ToF sensor to be fully operational and the data line (PB3) to be connected securely, or the ESM system may become trapped or significantly delayed by an inability to initialize or communicate with the MC. This error was predominantly experienced in testing with poor/degraded wires. This bottleneck was confirmed by testing different code snippets to rule out coding errors. Replacing low-quality wires with higher-quality wires resolved the issue temporarily.

11 – References and Additional Reading

11.1 – Additional Reading

Please note that all electronic component information is sourced from the component's respective data sheets.

[1] Texas Insruments, "MSP432E401Y SimpleLink™ Ethernet Microcontroller," Oct. 2017. Available: https://www.ti.com/lit/gpn/MSP432E401Y

[2] Arduino, "VMA401," Oct. 2018. Available: https://descargas.cetronic.es/VMA401.pdf

[3] STMicroelectronics, "VL53L1X," Aug. 2024. Available: https://www.st.com/resource/en/datasheet/vl53l1x.pdf

11.2 – References

[1] "Floating-Point Numbers - MATLAB & Simulink," *www.mathworks.com*. https://www.mathworks.com/help/matlab/matlab_prog/floating-point-numbers.html